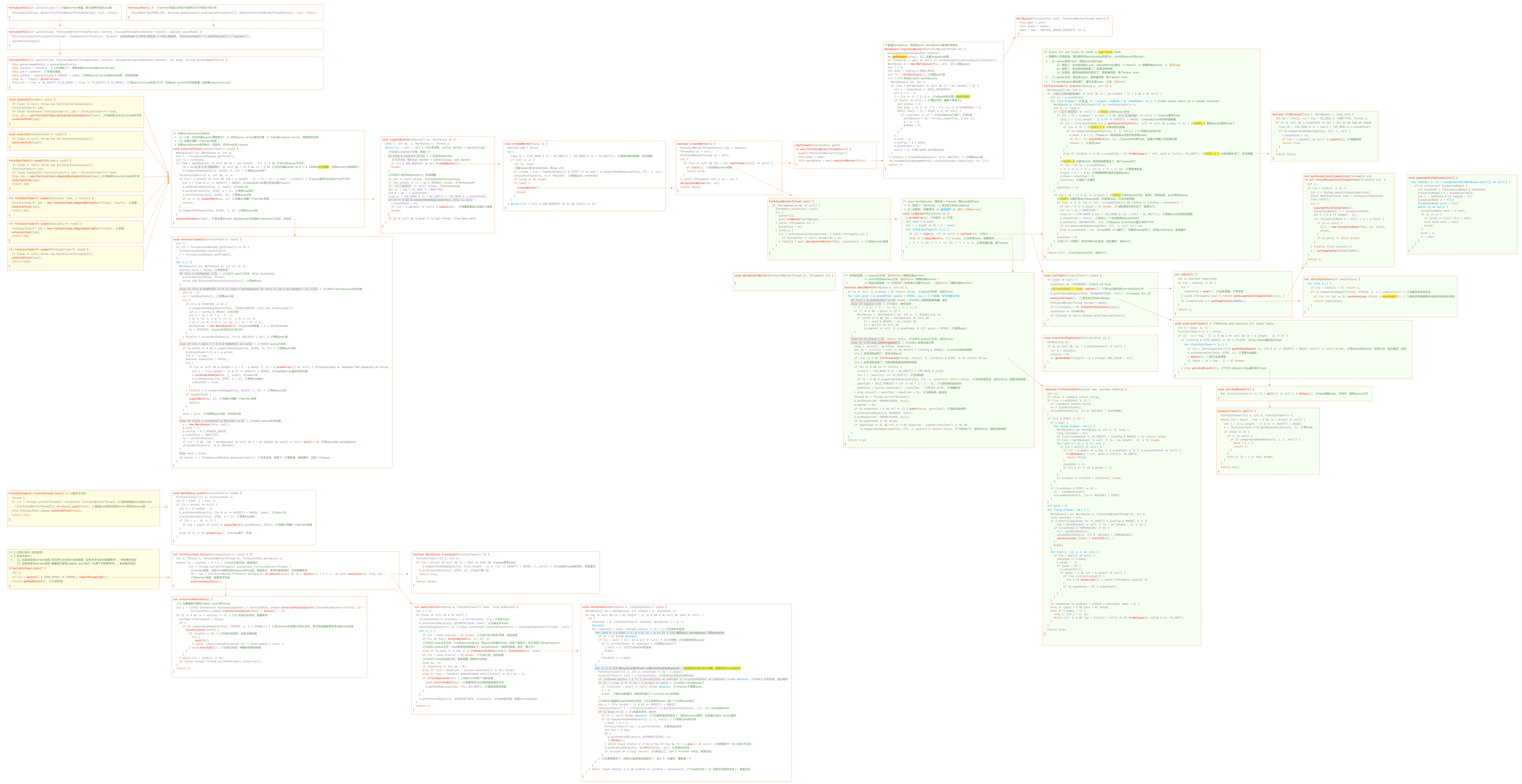

```
ForkJoinPool(int parallelism) { //指定worker数量，默认使用系统的cpu数
  this(parallelism, defaultForkJoinWorkerThreadFactory, null, false);
}
```

```
ForkJoinPool() {  //worker数量从系统CPU数和32767间选个较小的
  this(Math.min(MAX_CAP, Runtime.getRuntime().availableProcessors()), defaultForkJoinWorkerThreadFactory, null, false);
}
```

```
ForkJoinPool(int parallelism, ForkJoinWorkerThreadFactory factory, UncaughtExceptionHandler handler, boolean asyncMode) {
  this(checkParallelism(parallelism), checkFactory(factory), handler, asyncMode ? FIFO_QUEUE : LIFO_QUEUE, "ForkJoinPool-" + nextPoolId() + "-worker-");
  checkPermission();
}
```

```
ForkJoinPool(int parallelism, ForkJoinWorkerThreadFactory factory, UncaughtExceptionHandler handler, int mode, String workerNamePrefix) {
  this.workerNamePrefix = workerNamePrefix;
  this.factory = factory; //工作线程工厂，用来创建ForkJoinWorkerThread
  this.ueh = handler; //异常处理器
  this.config = (parallelism & SMASK) | mode; //保存parallelism和mode信息，供后续读取
  long np = (long)(-parallelism);
  this.ctl = ((np << AC_SHIFT) & AC_MASK) | ((np << TC_SHIFT) & TC_MASK); //将parallelism存到ctl中，后续add worker时控制数量（线程数=parallelism）
}
```

```
void execute(Runnable task) {
  if (task == null) throw new NullPointerException();
  ForkJoinTask<?> job;
  if (task instanceof ForkJoinTask<?>) job = (ForkJoinTask<?>) task;
  else job = new ForkJoinTask.RunnableExecuteAction(task); //封装成ForkJoinTask的子类
  externalPush(job);
}
```

```
void execute(ForkJoinTask<?> task) {
  if (task == null) throw new NullPointerException();
  externalPush(task);
}
```

```
ForkJoinTask<?> submit(Runnable task) {
  if (task == null) throw new NullPointerException();
  ForkJoinTask<?> job;
  if (task instanceof ForkJoinTask<?>) job = (ForkJoinTask<?>) task;
  else job = new ForkJoinTask.AdaptedRunnableAction(task);//封装成ForkJoinTask的子类
  externalPush(job);
  return job;
}
```

```
<T> ForkJoinTask<T> submit(Runnable task, T result) {
  ForkJoinTask<T> job = new ForkJoinTask.AdaptedRunnable<T>(task, result); //封装
  externalPush(job);
  return job;
}
```

```
<T> ForkJoinTask<T> submit(Callable<T> task) {
  ForkJoinTask<T> job = new ForkJoinTask.AdaptedCallable<T>(task); //封装
  externalPush(job);
  return job;
}
```

```
<T> ForkJoinTask<T> submit(ForkJoinTask<T> task) {
  if (task == null) throw new NullPointerException();
  externalPush(task);
  return task;
}
```

```
/* 如果workQueues已初始化：
 *  1).入队：定位所属queue(偶数索引) → 定位queue.array里的位置 → task放入queue.array，更新相关信息
 *  2).创建/唤醒一个worker线程
 * 如果workQueues未初始化：初始化，并将task存入queue
void externalPush(ForkJoinTask<?> task) {
  WorkQueue[] ws; WorkQueue q; int m;
  int r = ThreadLocalRandom.getProbe();
  int rs = runState;
  if ((ws = workQueues) ≠ null && (m = (ws.length - 1)) ≥ 0 && //workQueues不为空
      (q = ws[m & r & SQMASK]) ≠ null && r ≠ 0 && rs > 0 && //定位所属queue (m & r & SQMASK必为偶数，没和worker线程绑定)
      U.compareAndSwapInt(q, QLOCK, 0, 1)) { //拿到queue锁了
    ForkJoinTask<?>[] a; int am, n, s;
    if ((a = q.array) ≠ null && (am = a.length - 1) > (n = (s = q.top) - q.base)) { //queue里放task的array不为空
      int j = ((am & s) << ASHIFT) + ABASE; //task在array里的存放位置(top+1)
      U.putOrderedObject(a, j, task); //task入队
      U.putOrderedInt(q, QTOP, s + 1); //更新top指针
      U.putIntVolatile(q, QLOCK, 0); //释放queue锁
      if (n ≤ 1) signalWork(ws, q); //创建or唤醒一个worker线程
      return;
    }
    U.compareAndSwapInt(q, QLOCK, 1, 0); //释放queue锁
  }
  externalSubmit(task); //首次提交task，WorkQueue为空或WorkQueue[I]为空，初始化
}
```

```
void externalSubmit(ForkJoinTask<?> task) {
  int r;
  if ((r = ThreadLocalRandom.getProbe()) == 0) {
    ThreadLocalRandom.localInit();
    r = ThreadLocalRandom.getProbe();
  }
  for (;;) {
    WorkQueue[] ws; WorkQueue q; int rs, m, k;
    boolean move = false; //竞争标志
    if ((rs = runState) < 0) { //CASE1:pool已关闭，help terminate
      tryTerminate(false, false);
      throw new RejectedExecutionException(); //拒绝task
    }
    else if ((rs & STARTED) == 0 || ((ws = workQueues) == null || (m = ws.length - 1) < 0)) { //CASE2:workQueues还未创建
      int ns = 0;
      rs = lockRunState(); //获取pool锁
      try {
        if ((rs & STARTED) == 0) {
          U.compareAndSwapObject(this, STEALCOUNTER, null,new AtomicLong());
          int p = config & SMASK; //并行数
          int n = (p > 1) ? p - 1 : 1;
          n |= n >>> 1; n |= n >>> 2;  n |= n >>> 4;
          n |= n >>> 8; n |= n >>> 16; n = (n + 1) << 1;
          workQueues = new WorkQueue[n]; //queues的数量 = 2 * parallelism
          ns = STARTED; //pool状态改为STARTED
        }
      } finally { unlockRunState(rs, (rs & ~RSLOCK) | ns); } //释放pool锁
    }
    else if ((q = ws[k = r & m & SQMASK]) ≠ null) { //CASE3:queue已存在
      if (q.qlock == 0 && U.compareAndSwapInt(q, QLOCK, 0, 1)) { //获取queue锁
        ForkJoinTask<?>[] a = q.array;
        int s = q.top;
        boolean submitted = false;
        try {
          if ((a ≠ null && a.length > s + 1 - q.base) || (a = q.growArray()) ≠ null) { //Initializes or doubles the capacity of array.
            int j = (((a.length - 1) & s) << ASHIFT) + ABASE; //task在array里的存放位置
            U.putOrderedObject(a, j, task); //task入队
            U.putOrderedInt(q, QTOP, s + 1); //更新top指针
            submitted = true;
          }
        } finally { U.compareAndSwapInt(q, QLOCK, 1, 0); } //释放queue锁
        if (submitted) {
          signalWork(ws, q); //创建or唤醒一个worker线程
          return;
        }
      }
      move = true; //没拿到queue锁，有并发冲突
    }
    else if (((rs = runState) & RSLOCK) == 0) { //CASE4:queue还未创建
      q = new WorkQueue(this, null);
      q.hint = r;
      q.config = k | SHARED_QUEUE;
      q.scanState = INACTIVE;
      rs = lockRunState();
      if (rs > 0 &&  (ws = workQueues) ≠ null && k < ws.length && ws[k] == null) ws[k] = q; //新queue加入workQueues
      unlockRunState(rs, rs & ~RSLOCK);
    }
    else move = true;
    if (move) r = ThreadLocalRandom.advanceProbe(r); //并发冲突，获取下一个随机值，继续循环，定位一个queue
  }
}
```

```
void signalWork(WorkQueue[] ws, WorkQueue q) {
  long c; int sp, i; WorkQueue v; Thread p;
  while ((c = ctl) < 0L) { //AC是负数，active worker < parallelism
    //CASE1:workers不够，增加一个
    if ((sp = (int)c) == 0) { //没有空闲worker
      //TC不为0，即total worker < parallelism, add worker
      if ((c & ADD_WORKER) ≠ 0L) tryAddWorker(c);
      break;
    }
    //CASE2:有空闲的workers，尝试唤醒
    if (ws == null) break; //unstarted/terminated
    if (ws.length ≤ (i = sp & SMASK)) break; //terminated
    if ((v = ws[i]) == null) break; //terminating
    int vs = (sp + SS_SEQ) & ~INACTIVE;
    int d = sp - v.scanState;
    long nc = (UC_MASK & (c + AC_UNIT)) | (SP_MASK & v.stackPred);
    if (d == 0 && U.compareAndSwapLong(this, CTL, c, nc)) {
      v.scanState = vs;
      if ((p = v.parker) ≠ null) U.unpark(p);  //唤醒阻塞链上的第1个线程
      break;
    }
    if (q ≠ null && q.base == q.top) break; //no more work
  }
}
```

```
void tryAddWorker(long c) {
  boolean add = false;
  do {
    long nc = ((AC_MASK & (c + AC_UNIT)) | (TC_MASK & (c + TC_UNIT))); //更新活跃线程数、总线程数
    if (ctl == c) {
      int rs, stop;
      //确认pool没terminated，并cas ctl
      if ((stop = (rs = lockRunState()) & STOP) == 0) add = U.compareAndSwapLong(this, CTL, c, nc);
      unlockRunState(rs, rs & ~RSLOCK); //解锁pool.runState
      if (stop ≠ 0) break;
      if (add) {
        createWorker();
        break;
      }
    }
  } while (((c = ctl) & ADD_WORKER) ≠ 0L && (int)c == 0);
}
```

```
boolean createWorker() {
  ForkJoinWorkerThreadFactory fac = factory;
  Throwable ex = null;
  ForkJoinWorkerThread wt = null;
  try {
    if (fac ≠ null && (wt = fac.newThread(this)) ≠ null) {
      wt.start(); //启动新worker线程
      return true;
    }
  } catch (Throwable rex) { ex = rex }
  deregisterWorker(wt, ex);
  return false;
}
```

```
newThread(ForkJoinPool pool)
→ new ForkJoinWorkerThread(pool) {
  super("aForkJoinWorkerThread");
  this.pool = pool;
  this.workQueue = pool.registerWorker(this);
}
```

```
//新建workQueue，放到在pool.workQueues数组的奇数位
WorkQueue registerWorker(ForkJoinWorkerThread wt) {
  UncaughtExceptionHandler handler;
  wt.setDaemon(true); //1.设置为deamon线程
  if ((handler = ueh) ≠ null) wt.setUncaughtExceptionHandler(handler);
  WorkQueue w = new WorkQueue(this, wt); //2.分配queue
  int i = 0;
  int mode = config & MODE_MASK;
  int rs = lockRunState(); //获取pool锁
  try { //3.将w加入pool.workQueues
    WorkQueue[] ws; int n;
    if ((ws = workQueues) ≠ null && (n = ws.length) > 0) {
      int s = indexSeed += SEED_INCREMENT;
      int m = n - 1;
      i = ((s << 1) | 1) & m; //w在ws中的位置（始终为奇数）
      if (ws[i] ≠ null) { //槽位冲突，重新计算索引i
        int probes = 0;
        int step = (n ≤ 4) ? 2 : ((n >>> 1) & EVENMASK) + 2;
        while (ws[i = (i + step) & m] ≠ null) {
          if (++probes ≥ n) { //workQueue[]满了，扩容2倍
            workQueues = ws = Arrays.copyOf(ws, n <<= 1);
            m = n - 1;
            probes = 0;
          }
        }
      }
      w.hint = s;
      w.config = i | mode;
      w.scanState = i;
      ws[i] = w; //加入pool.workQueues
    }
  } finally { unlockRunState(rs, rs & ~RSLOCK); } //释放pool锁
  wt.setName(workerNamePrefix.concat(Integer.toString(i >>> 1)));
  return w;
}
```

```
WorkQueue(ForkJoinPool pool, ForkJoinWorkerThread owner) {
  this.pool = pool;
  this.owner = owner;
  base = top = INITIAL_QUEUE_CAPACITY >>> 1;
}
```

```
ForkJoinWorkerThread.run() {
  if (workQueue.array == null) {
    Throwable exception = null;
    try {
      onStart();
      pool.runWorker(workQueue);
    } catch (Throwable ex) {
      exception = ex;
    } finally {
      try { onTermination(exception); } catch (Throwable ex) {
        if (exception == null) exception = ex;
      } finally { pool.deregisterWorker(this, exception); } //注销worker线程
    }
  }
}
```

```
/* scan workQueues: 随机选一个queue，取base处的task
 * 1).取到了：执行task → 清空自己的workQueue
 * 2).没取到：阻塞等待 → 继续循环 or 返回（注销worker)
void runWorker(WorkQueue w) {
  w.growArray(); //初始化 or 扩容
  int seed = w.hint;
  int r = (seed == 0) ? 1 : seed;
  for (ForkJoinTask<?> t;;) {
    if ((t = scan(w, r)) ≠ null) w.runTask(t); //执行
    else if (!awaitWork(w, r)) break; //没发现task，阻塞等待
    r ^= r << 13; r ^= r >>> 17; r ^= r << 5; //修改随机值，换个queue
  }
}
```

```
/* Scans for and tries to steal a top-level task.
 * 根据传入的随机数，算出随机的workQueues的索引k，从k处的queue开始scan:
 *
 *   1).queue里有task，读取base处的task
 *      a).读到了，且当前线程alive: base处的task置空 → base+1 → 唤醒其他worker → 返回task
 *      b).读到了，但当前线程阻塞了：如果当前线程
 *      c).没读到，被其他线程抢先拿走了：更新随机数，换个queue scan
 *
 *   2).queue为空，或没有task：更新随机数，换个queue scan
 *   3).workQueues里找遍了，都没发现task：灭活，返回null
ForkJoinTask<?> scan(WorkQueue w, int r) {
  WorkQueue[] ws; int m;
  if ((ws = workQueues) ≠ null && (m = ws.length - 1) > 0 && w ≠ null) {
    int ss = w.scanState;
    for (int origin = r & m, k = origin, oldSum = 0, checkSum = 0;;) { //r&m: scans start at a random location
      WorkQueue q; ForkJoinTask<?>[] a; ForkJoinTask<?> t;
      int b, n; long c;
      if ((q = ws[k]) ≠ null) { //CASE1:k处的queue非空
        if ((n = (b = q.base) - q.top) < 0 && (a = q.array) ≠ null) { //queue里有task
          long i = (((a.length - 1) & b) << ASHIFT) + ABASE; //base处task的内存偏移量
          if ((t = ((ForkJoinTask<?>) U.getObjectVolatile(a, i))) ≠ null && q.base == b) { //CASE1.1:拿到base位置的task了
            if (ss ≥ 0) { //CASE1.1.1:当前线程活跃着
              if (U.compareAndSwapObject(a, i, t, null)) { //获取task执行权
                q.base = b + 1; //base+1 (其他线程从这里开始获取task)
                if (n < -1) signalWork(ws, q); //queue中还有task，创建or唤醒工作线程处理
                return t; //返回task
              }
            }
            else if (oldSum == 0 && w.scanState < 0) tryRelease(c = ctl, ws[m & (int)c], AC_UNIT); //CASE1.1.2:当前线程失活了，尝试唤醒
          }
          //CASE1.2:没拿到task，被其他线程拿走了，换个queue试试
          if (ss < 0) ss = w.scanState;
          r ^= r << 1; r ^= r >>> 3; r ^= r << 10; //更新随机值
          origin = k = r & m; //根据新随机值定位新的queue
          oldSum = checkSum = 0;
          continue; //跳至下次循环
        }
        checkSum += b;
      }
      if ((k = (k + 1) & m) == origin) { //CASE2:k处的queue为空，更新k，继续自旋，scan其他queue
        //CASE3:扫描所有workQueues后，未发现task，灭活当前线程
        if ((ss ≥ 0 || (ss == (ss = w.scanState))) && oldSum == (oldSum = checkSum)) {
          if (ss < 0 || w.qlock < 0) break; //当前线程已经失活了，直接null
          int ns = ss | INACTIVE;
          long nc = ((SP_MASK & ns) | (UC_MASK & ((c = ctl) - AC_UNIT))); //更新pool的活跃线程数
          w.stackPred = (int)c; //保存上一个失活线程的scanState
          U.putInt(w, QSCANSTATE, ns); //将queue.scanState置为INACTIVE
          if (U.compareAndSwapLong(this, CTL, c, nc)) ss = ns;
          else w.scanState = ss; //cas失败，ctl值变了，可能有task进来了，回滚scanState，继续循环
        }
        checkSum = 0;
      } //进入下一次循环，若还没有task金来，退出循环，返回null
    }
  }
  return null; //workQueues为空，返回null
}
```

```
boolean tryRelease(long c, WorkQueue v, long inc) {
  int sp = (int)c, vs = (sp + SS_SEQ) & ~INACTIVE; Thread p;
  if (v ≠ null && v.scanState == sp) { //v is at top of stack
    long nc = (UC_MASK & (c + inc)) | (SP_MASK & v.stackPred);
    if (U.compareAndSwapLong(this, CTL, c, nc)) {
      v.scanState = vs;
      if ((p = v.parker) ≠ null) U.unpark(p); //唤醒线程
      return true;
    }
  }
  return false;
}
```

```
void deregisterWorker(ForkJoinWorkerThread wt, Throwable ex) {

}
```

```
/* 先随机自旋 → queue已关闭，返回false（销毁当前worker)
             → pool中空闲worker太多，返回false（销毁当前worker ）
 *           → 挂起当前线程 → TIMEOUT（外部很久没提交task），返回fasle（销毁当前worker)
boolean awaitWork(WorkQueue w, int r) {
  if (w == null || w.qlock < 0) return false; //queue已关闭，返回false
  for (int pred = w.stackPred, spins = SPINS, ss;;) { //自旋，SPINS默认为0
    if ((ss = w.scanState) ≥ 0) break; //CASE1:当前线程被唤醒，退出
    else if (spins > 0) { //CASE2: 随机自旋
      r ^= r << 6; r ^= r >>> 21; r ^= r << 7;
      if (r ≥ 0 && --spins == 0) {
        WorkQueue v; WorkQueue[] ws; int s, j; AtomicLong sc;
        if (pred ≠ 0 && (ws = workQueues) ≠ null &&
            (j = pred & SMASK) < ws.length &&
            (v = ws[j]) ≠ null &&
            (v.parker == null || v.scanState ≥ 0)) spins = SPINS; //继续spin
      }
    }
    else if (w.qlock < 0) return fasle; //CASE3:queue已关闭，返回false
    else if (!Thread.interrupted()) { //CASE4:线程没被打断
      long c, prevctl, parkTime, deadline;
      int ac = (int)((c = ctl) >> AC_SHIFT) + (config & SMASK); //pool的活跃线程数
      //4.1.没有活跃线程了，尝试关闭pool
      if ((ac ≤ 0 && tryTerminate(false, false)) || (runState & STOP) ≠ 0) return false;
      //4.2.没有活跃线程了，当前线程是最后挂起的线程
      if (ac ≤ 0 && ss == (int)c) {
        prevctl = (UC_MASK & (c + AC_UNIT)) | (SP_MASK & pred);
        int t = (short)(c >>> TC_SHIFT); //总线程数
        if (t > 2 && U.compareAndSwapLong(this, CTL, c, prevctl)) return false; //空闲线程较多，返回false，销毁当前线程
        parkTime = IDLE_TIMEOUT * ((t ≥ 0) ? 1 : 1 - t); //当前线程挂起时间
        deadline = System.nanoTime() + parkTime - TIMEOUT_SLOP; //唤醒时间
      } else prevctl = parkTime = deadline = 0L; //当前线程一直挂起
      Thread wt = Thread.currentThread();
      U.putObject(wt, PARKBLOCKER, this);
      w.parker = wt;
      if (w.scanState < 0 && ctl == c) U.park(false, parkTime); //挂起当前线程
      U.putOrderedObject(w, QPARKER, null);
      U.putObject(wt, PARKBLOCKER, null);
      if (w.scanState ≥ 0) break;
      if (parkTime ≠ 0L && ctl == c && deadline - System.nanoTime() ≤ 0L &&
          U.compareAndSwapLong(this, CTL, c, prevctl)) return false; //TIMEOUT了，返回false，销毁当前线程
    }
  }
  return true;
}
```

```
void runTask(ForkJoinTask<?> task) {
  if (task ≠ null) {
    scanState &= ~SCANNING; //mark as busy
    (currentSteal = task).doExec(); //将task暂存到currentSteal中
    U.putOrderedObject(this, QCURRENTSTEAL, null); //release for GC
    execLocalTasks(); //清空自己的workQueue
    ForkJoinWorkerThread thread = owner;
    if (++nsteals < 0) transferStealCount(pool);
    scanState |= SCANNING;
    if (thread ≠ null) thread.afterTopLevelExec();
  }
}
```

```
int doExec() {
  int s; boolean completed;
  if ((s = status) ≥ 0) {
    try {
      completed = exec(); //业务逻辑，子类实现
    } catch (Throwable rex) { return setExceptionalCompletion(rex); }
    if (completed) s = setCompletion(NORMAL);
  }
  return s;
}
```

```
int setExceptionalCompletion(Throwable ex)
→ int recordExceptionalCompletion(Throwable ex)  {
  int s;
  if ((s = status) ≥ 0) {
    int h = System.identityHashCode(this);
    final ReentrantLock lock = exceptionTableLock;
    lock.lock();
    try {
      expungeStaleExceptions();
      ExceptionNode[] t = exceptionTable;
      int i = h & (t.length - 1);
      for (ExceptionNode e = t[i]; ; e = e.next) {
        if (e == null) {
          t[i] = new ExceptionNode(this, ex, t[i]);
          break;
        }
        if (e.get() == this) break;
      }
    } finally {lock.unlock();}
    s = setCompletion(EXCEPTIONAL);
  }
  return s;
}
```

```
void expungeStaleExceptions() {
  for (Object x; (x = exceptionTableRefQueue.poll()) ≠ null;) {
    if (x instanceof ExceptionNode) {
      int hashCode = ((ExceptionNode)x).hashCode;
      ExceptionNode[] t = exceptionTable;
      int i = hashCode & (t.length - 1);
      ExceptionNode e = t[i];
      ExceptionNode pred = null;
      while (e ≠ null) {
        ExceptionNode next = e.next;
        if (e == x) {
          if (pred == null) t[i] = next;
          else pred.next = next;
          break;
        }
        pred = e;
        e = next;
      }
    }
  }
}
```

```
int setCompletion(int completion) {
  for (int s;;) {
    if ((s = status) < 0) return s;
    if (U.compareAndSwapInt(this, STATUS, s, s | completion)) { //设置任务完成标志
      if ((s >>> 16) ≠ 0) synchronized (this) { notifyAll(); } //通知所有阻塞等待当前任务完成的线程
      return completion;
    }
  }
}
```

```
void transferStealCount(ForkJoinPool p) {
  AtomicLong sc;
  if (p ≠ null && (sc = p.stealCounter) ≠ null) {
    int s = nsteals;
    nsteals = 0;
    sc.getAndAdd((long)(s < 0 ? Integer.MAX_VALUE : s));
  }
}
```

```
void execLocalTasks() { //Removes and executes all local tasks.
  int b = base, m, s;
  ForkJoinTask<?>[] a = array;
  if (b - (s = top - 1) ≤ 0 && a ≠ null && (m = a.length - 1) ≥ 0) {
    if ((config & FIFO_QUEUE) == 0) { //LIFO, 从top→base遍历执行task
      for (ForkJoinTask<?> t;;) {
        if ((t = (ForkJoinTask<?>)U.getAndSetObject (a, ((m & s) << ASHIFT) + ABASE, null)) == null) break; //取出top处的task，如果为空，跳出循环，返回
        U.putOrderedInt(this, QTOP, s); //更新top指针
        t.doExec(); //执行业务逻辑
        if (base - (s = top - 1) > 0) break;
      }
    } else pollAndExecAll(); //FIFO:从base→top遍历执行task
  }
}
```

```
void pollAndExecAll() {
  for (ForkJoinTask<?> t; (t = poll()) ≠ null;) t.doExec(); //base端取task，并执行，直到queue为空
}
```

```
ForkJoinTask<?> poll() {
  ForkJoinTask<?>[] a; int b; ForkJoinTask<?> t;
  while ((b = base) - top < 0 && (a = array) ≠ null) {
    int j = (((a.length - 1) & b) << ASHIFT) + ABASE;
    t = (ForkJoinTask<?>)U.getObjectVolatile(a, j); //取task
    if (base == b) {
      if (t ≠ null) {
        if (U.compareAndSwapObject(a, j, t, null)) {
          base = b + 1;
          return t;
        }
      }
      else if (b + 1 == top) break;
    }
  }
  return null;
}
```

```
boolean tryTerminate(boolean now, boolean enable) {
  int rs;
  if (this == common) return false;
  if ((rs = runState) ≥ 0) {
    if (!enable) return false;
    rs = lockRunState();
    unlockRunState(rs, (rs & ~RSLOCK) | SHUTDOWN);
  }
  if ((rs & STOP) == 0) {
    if (!now) {
      for (long oldSum = 0L;;) {
        WorkQueue[] ws; WorkQueue w; int m, b; long c;
        long checkSum = ctl;
        if ((int)(checkSum >> AC_SHIFT) + (config & SMASK) > 0) return false;
        if ((ws = workQueues) == null || (m = ws.length - 1) ≤ 0) break;
        for (int i = 0; i ≤ m; ++i) {
          if ((w = ws[i]) ≠ null) {
            if ((b = w.base) ≠ w.top || w.scanState ≥ 0 || w.currentSteal ≠ null) {
              tryRelease(c = ctl, ws[m & (int)c], AC_UNIT);
              return false;
            }
            checkSum += b;
            if ((i & 1) == 0) w.qlock = -1;
          }
        }
        if (oldSum == (oldSum = checkSum)) break;
      }
    }
    if ((runState & STOP) == 0) {
      rs = lockRunState();
      unlockRunState(rs, (rs & ~RSLOCK) | STOP);
    }
  }
  int pass = 0;
  for (long oldSum = 0L;;) {
    WorkQueue[] ws; WorkQueue w; ForkJoinWorkerThread wt; int m;
    long checkSum = ctl;
    if ((short)(checkSum >>> TC_SHIFT) + (config & SMASK) ≤ 0 ||
        (ws = workQueues) == null || (m = ws.length - 1) ≤ 0) {
      if ((runState & TERMINATED) == 0) {
        rs = lockRunState();
        unlockRunState(rs, (rs & ~RSLOCK) | TERMINATED);
        synchronized (this) { notifyAll(); }
      }
      break;
    }
    for (int i = 0; i ≤ m; ++i) {
      if ((w = ws[i]) ≠ null) {
        checkSum += w.base;
        w.qlock = -1;
        if (pass > 0) {
          w.cancelAll();
          if (pass > 1 && (wt = w.owner) ≠ null) {
            if (!wt.isInterrupted()) {
              try { wt.interrupt(); } catch (Throwable ignore) {}
            }
            if (w.scanState < 0) U.unpark(wt);
          }
        }
      }
    }
    if (checkSum ≠ oldSum) { oldSum = checkSum; pass = 0; }
    else if (pass > 3 && pass > m) break;
    else if (++pass > 1) {
      long c; int j = 0, sp;
      while (j++ ≤ m && (sp = (int)(c = ctl)) ≠ 0) tryRelease(c, ws[sp & m], AC_UNIT);
    }
  }
  return true;
}
```

```
ForkJoinTask<V> ForkJoinTask.fork() { //提交子任务
  Thread t;
  if ((t = Thread.currentThread()) instanceof ForkJoinWorkerThread) //当前线程是pool的worker
    ((ForkJoinWorkerThread)t).workQueue.push(this); //直接push到和当前worker绑定的queue里
  else ForkJoinPool.common.externalPush(this);
  return this;
}
```

```
void WorkQueue.push(ForkJoinTask<?> task) {
  ForkJoinTask<?>[] a; ForkJoinPool p;
  int b = base, s = top, n;
  if ((a = array) ≠ null) {
    int m = a.length - 1;
    U.putOrderedObject(a, ((m & s) << ASHIFT) + ABASE, task); //task入队
    U.putOrderedInt(this, QTOP, s + 1); //更新top指针
    if ((n = s - b) ≤ 1) {
      if ((p = pool) ≠ null) p.signalWork(p.workQueues, this); //创建or唤醒一个worker线程
    }
    else if (n ≥ m) growArray(); //array满了，扩容
  }
}
```

```
/* 1.任务已执行:返回结果
 * 2.任务未执行:
 *   1).当前线程是worker线程:尝试参与任务执行加快速度（没有多余任务时阻塞等待），有结果时返回
     2).当前线程非worker线程:看看能不能用common pool执行（如果不行阻塞等待），有结果时返回
V ForkJoinTask.join() {
  int s;
  if ((s = doJoin() & DONE_MASK) ≠ NORMAL) reportException(s);
  return getRawResult(); //子类实现
}
```

```
int ForkJoinTask.doJoin(ForkJoinTask<?> task) { //
  int s; Thread t; ForkJoinWorkerThread wt; ForkJoinPool.WorkQueue w;
  return (s = status) < 0 ? s : //task已执行完，直接返回
         ((t = Thread.currentThread()) instanceof ForkJoinWorkerThread) ?
          //worker线程：当前task刚好在自己queue的top处，直接执行，若成功直接返回；否则阻塞等待
          (w = (wt = (ForkJoinWorkerThread)t).workQueue).tryUnpush(this) && (s = doExec()) < 0 ? s : wt.pool.awaitJoin(w, this, 0L) :
          //非worker线程：阻塞等待完成
          externalAwaitDone();
}
```

```
boolean WorkQueue.tryUnpush(ForkJoinTask<?> t) {
  ForkJoinTask<?>[] a; int s;
  if ((a = array) ≠ null && (s = top) ≠ base && //queue里有task
      U.compareAndSwapObject(a, (((a.length - 1) & --s) << ASHIFT) + ABASE, t, null)) { //top处的task刚好是t，将其置空
    U.putOrderedInt(this, QTOP, s); //top下移一位
    return true;
  }
  return false;
}
```

```
int externalAwaitDone() {
  //1.先看看能不能用common pool执行task
  int s = ((this instanceof CountedCompleter) ? ForkJoinPool.common.externalHelpComplete((CountedCompleter<?>)this, 0) :
           ForkJoinPool.common.tryExternalUnpush(this) ? doExec() : 0);
  if (s ≥ 0 && (s = status) >= 0) { //2.任务还未完成，阻塞等待
    boolean interrupted = false;
    do {
      if (U.compareAndSwapInt(this, STATUS, s, s | SIGNAL)) { //在status中设置SIGNAL信号，表示有线程阻塞等待当前task完成
        synchronized (this) {
          if (status >= 0) { //任务未完成时，挂起当前线程
            try {
              wait(0L);
            } catch (InterruptedException ie) { interrupted = true; }
          } else notifyAll(); //任务已完成，唤醒所有等待线程
        }
      }
    } while ((s = status) ≥ 0);
    if (interrupted) Thread.currentThread().interrupt();
  }
  return s;
}
```

```
int awaitJoin(WorkQueue w, ForkJoinTask<?> task, long deadline) {
  int s = 0;
  if (task ≠ null && w ≠ null) {
    ForkJoinTask<?> prevJoin = w.currentJoin; //上一个合并task
    U.putOrderedObject(w, QCURRENTJOIN, task); //当前合并task
    CountedCompleter<?> cc = (task instanceof CountedCompleter) ? (CountedCompleter<?>)task : null;
    for (;;) {
      if ((s = task.status) < 0) break; //已执行完|取消|异常，返回结果
      if (cc ≠ null) helpComplete(w, cc, 0); //
      //CASE1:queue不为空：tryRemoveAndExec（在queue中查找task，找到了就执行，找不到进入helpStealer）
      //CASE2:queue为空：task被其他线程偷走了，helpStealer（找到窃取者，助它一臂之力）
      else if (w.base == w.top || w.tryRemoveAndExec(task)) helpStealer(w, task);
      if ((s = task.status) < 0) break; //已执行完，返回结果
      //CASE3:task还没执行完，准备阻塞（释放CPU资源）
      long ms, ns;
      if (deadline == 0L) ms = 0L;
      else if ((ns = deadline - System.nanoTime()) ≤ 0L) break;
      else if ((ms = TimeUnit.NANOSECONDS.toMillis(ns)) ≤ 0L) ms = 1L;
      if (tryCompensate(w)) { //给pool补偿1个活跃线程
        task.internalWait(ms); //阻塞等待task被其他线程执行完
        U.getAndAddLong(this, CTL, AC_UNIT); //更新活跃线程数
      }
    }
    U.putOrderedObject(w, QCURRENTJOIN, prevJoin); //task执行完，恢复currentJoin
  }
  return s;
}
```

```
void helpStealer(WorkQueue w, ForkJoinTask<?> task) {
  WorkQueue[] ws = workQueues; int oldSum = 0, checkSum, m;
  if (ws ≠ null && (m = ws.length - 1) ≥ 0 && w ≠ null && task ≠ null) {
    do {
      checkSum = 0; ForkJoinTask<?> subtask; WorkQueue j = w, v;
      descent:
      for (subtask = task; subtask.status >= 0; ) { //任务还未完成
        for (int h = j.hint | 1, k = 0, i; ; k += 2) { //1.遍历pool.workQueues，寻找stealer
          if (k > m) break descent;
          if ((v = ws[i = (h + k) & m]) ≠ null) { //I为奇数，v为当前查找的queue
            if (v.currentSteal == subtask) { //找到stealer了
              j.hint = I; //记下stealer的坐标
              break;
            }
            checkSum += v.base;
          }
        }
        for (;;) { //2.帮stealer执行task（v是stealer的私有queue），加快整体任务的执行进度，缩短等待join的时间
          ForkJoinTask<?>[] a; int b; checkSum += (b = v.base);
          ForkJoinTask<?> next = v.currentJoin; //stealer正在join的task
          if (subtask.status < 0 || j.currentJoin ≠ subtask || v.currentSteal ≠ subtask) break descent; //CASE1:任务完成，退出循环
          if (b - v.top ≥ 0 || (a = v.array) == null) { //CASE2:v中没有task了
            if ((subtask = next) == null) break descent; //stealer不需要join
            j = v;
            break; //跳出当前循环，继续寻找偷了v.currentJoin的线程
          }
          //CASE3:被偷的task还未执行完成，v中又有其他task：偷一个v中的task执行
          int i = (((a.length - 1) & b) << ASHIFT) + ABASE;
          ForkJoinTask<?> t = ((ForkJoinTask<?>) U.getObjectVolatile(a, i)); //v base处的task
          if (v.base == b) { //v栈底无变化，执行t
            if (t == null) break descent; //t已被其他线程取走了，跳出descent循环，回到最外层do while循环
            if (U.compareAndSwapObject(a, i, t, null)) { //获取task执行权
              v.base = b + 1;
              ForkJoinTask<?> ps = w.currentSteal; //暂存w的状态
              int top = w.top;
              do {
                U.putOrderedObject(w, QCURRENTSTEAL, t);
                t.doExec();
              } while (task.status ≥ 0 && w.top ≠ top && (t = w.pop()) ≠ null); //继续执行t fork出的子任务
              U.putOrderedObject(w, QCURRENTSTEAL, ps); //恢复w的状态
              if (w.base ≠ w.top) return; //w来活儿了，can't further help，直接返回
            }
          } //如果栈底变了，说明t已被其他线程取走了，进入下一次循环，重新偷一个
        }
      }
    } while (task.status ≥ 0 && oldSum ≠ (oldSum = checkSum)); //task执行完了 or 窃取不到其他任务了，直接返回
  }
}
```